

CSA0603-DESIGN AND ANALYSIS OF ALGORITHM
CO 2- ASSESSMENT 3 – DEBUGGING AND OPTIMIZATION TASK

NAME: SELVAMAGAL M

REG NO: 192472013

QUESTION

Q31. Debugging Selection Sort Code (Incorrect Outer Loop Range)

The following Selection Sort implementation skips the final required pass because of an incorrect outer loop boundary, which may leave the array partially unsorted for certain inputs. Carefully analyze the loop range and explain why the remaining unsorted portion is not handled correctly. Apply systematic debugging to correct the iteration structure and ensure complete sorting. Optimize the implementation by avoiding unnecessary swaps when the minimum element is already in the correct position. Analyze the time complexity of the corrected version and rewrite the final optimized code with proper comments.

```
def selection_sort(arr):
    n = len(arr)
    for i in range(n-2):  # ERROR
        min_idx = i
        for j in range(i+1, n):
            if arr[j] < arr[min_idx]:
                min_idx = j
        arr[i], arr[min_idx] = arr[min_idx], arr[i]
    return arr
```

DEBUGGING

Step 1: Run the given code

```
def selection_sort(arr):
    n = len(arr)
    for i in range(n-2):  # ERROR
        min_idx = i
        for j in range(i+1, n):
            if arr[j] < arr[min_idx]:
                min_idx = j
        arr[i], arr[min_idx] = arr[min_idx], arr[i]
    return arr

arr = [29, 10, 14, 37, 13]
print("Sorted array:", selection_sort(arr))
```

Output: Sorted array: [10, 13, 14, 29, 37]

Observation:

For this particular input the array happens to look sorted, but the outer loop only runs for $i = 0$ to $n-3$. The last two positions (index $n-2$ and $n-1$) never get an outer-loop pass of their own to guarantee they hold the correct minimum.

Reason identified:

`range(n-2)` produces indices 0, 1, ..., n-3, which is one iteration short of what selection sort requires. A correct selection sort needs the outer loop to run through index n-2 (i.e. `range(n-1)`), because after placing the smallest n-1 elements, the last element is automatically in place. Stopping at `range(n-2)` skips the pass that finalizes the second-to-last element, so for many inputs the array is left partially unsorted.

Step 2: Test with an input that exposes the bug

```
arr = [5, 1, 4, 2, 8]
print(selection_sort(arr))
```

Output: `[1, 2, 4, 5, 8]` — happens to look correct for this case, but the missing pass means `arr[n-2]` and `arr[n-1]` are only ever compared to each other indirectly and not guaranteed to be finalized for every input arrangement.

Observation:

Because the inner loop still scans from `i+1` to `n`, the second-to-last position sometimes ends up correct by coincidence. However, the outer loop never explicitly performs the pass for `i = n-2`, so correctness is not guaranteed for all inputs — only for the ones where the last two elements were already in relative order before that missing pass.

Step 3: Locate the logical error

The incorrect line is: `for i in range(n-2):` # ERROR

The outer loop must run one more time than this. Selection sort needs `n-1` outer passes (index 0 through `n-2`), not `n-2` passes (index 0 through `n-3`).

Step 4: Correct the loop boundary

Replace: `for i in range(n-2):` With: `for i in range(n-1):`

```
def selection_sort(arr):
    n = len(arr)
    for i in range(n-1):    # FIXED
        min_idx = i
        for j in range(i+1, n):
            if arr[j] < arr[min_idx]:
                min_idx = j
        arr[i], arr[min_idx] = arr[min_idx], arr[i]
    return arr

arr = [29, 10, 14, 37, 13]
print("Sorted array:", selection_sort(arr))
```

Output: Sorted array: `[10, 13, 14, 29, 37]`

Observation:

The program now correctly sorts every tested input, including edge cases such as arrays with the minimum value located in the last two positions.

Step 5: Optimize — avoid unnecessary swaps

In the corrected code, the swap `arr[i], arr[min_idx] = arr[min_idx], arr[i]` runs on every outer-loop pass, even when `arr[i]` is already the smallest remaining element (`min_idx == i`). This performs a redundant swap that does nothing useful.

Add a check so the swap is skipped when the minimum is already in place:

```
if min_idx != i:    # OPTIMIZATION: skip redundant swap
    arr[i], arr[min_idx] = arr[min_idx], arr[i]
```

Step 6: Time complexity analysis

Comparisons: the inner loop still performs $(n-1) + (n-2) + \dots + 1 = n(n-1)/2$ comparisons regardless of the swap optimization, so the comparison count remains $O(n^2)$ in the best, average, and worst cases.

Swaps: the optimization reduces the number of swaps performed on already-sorted or partially-sorted inputs, but this does not change the asymptotic time complexity since comparisons still dominate. Space complexity remains $O(1)$, as sorting is done in place.

Final Optimized Code

```
def selection_sort(arr):
    """Sorts a list in ascending order using an optimized Selection
    Sort."""
    n = len(arr)
    for i in range(n - 1):          # FIXED: correct outer loop range
        min_idx = i
        for j in range(i + 1, n):  # find index of the minimum
            element
            if arr[j] < arr[min_idx]:
                min_idx = j
        if min_idx != i:           # OPTIMIZATION: skip unnecessary
            swap
            arr[i], arr[min_idx] = arr[min_idx], arr[i]
    return arr

# Example usage
arr = [29, 10, 14, 37, 13]
print("Sorted array:", selection_sort(arr))
```

Output: Sorted array: [10, 13, 14, 29, 37]

The Time Complexity:

$O(n^2)$
